

module-11-web-scraping

Chapter 11: Web Scraping | 34 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch11: webbrowser, requests, bs4, selenium

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Programs can browse: webbrowser opens pages; requests downloads them (raise_for_status() catches 404s); BeautifulSoup parses HTML so select() grabs elements by CSS selector and attrs like href/src yield data. Projects: mapit (address from clipboard/command line to maps), lucky-google (open top hits), xkcd downloader (crawl + save images + follow prev links). Selenium drives a REAL browser for forms, clicks, and special keys.

Key ideas to carry forward

- res = requests.get(url); res.raise_for_status(); open('f','wb').write(res.content) - the download skeleton.
- soup.select('.cover') / soup.select('#id') / elem.get('src') - CSS selection + attributes.
- Selenium: find_element(s), click(), send_keys(), Keys.DOWN/ENTER for keyboard.

Classic beginner mistakes

- ! Scraping without checking status/robots/terms - handle errors and respect sites.
- ! Brittle selectors that break when a site redesigns - select robustly.

Study order hint: 01-06 mapit, 07-12 requests, 13-20 bs4, 21-28 lucky + xkcd, 29-34 selenium in order.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. requests download skeleton?

A: `get(url) -> raise_for_status() -> write res.content (binary) or res.text`. Always check status; handle timeouts.

Q2. BeautifulSoup selection?

A: `Soup(html, 'html.parser')`; `select('.class')`/#id/tag names via CSS; `.get('href')/['src']` for attributes; `.getText()` for visible text.

Q3. How does the lucky-google flow work?

A: Build query URL from argv, download, select result links, open top N with webbrowser - CLI args to browser tabs.

Q4. xkcd crawler steps?

A: Download page, find comic img + prev link, save image, follow prev until start - loop with termination condition.

Q5. When do you need Selenium?

A: JavaScript-rendered pages, logins, clicks, form fills - a real browser you remote-control (`find_element`, `click`, `send_keys`, `Keys`).

Q6. Scraping ethics/errors?

A: Respect robots/terms, throttle requests, cache pages, expect layout changes - code defensively with checks.

Teach it to others

The one analogy that sticks

requests is ordering takeout (you get the bag = HTML); BeautifulSoup is unpacking and sorting the containers (select the noodles = CSS selector); Selenium is hiring a driver to sit at the restaurant and click the menu for you.

What your students will get wrong

- x Downloaded HTML equals what I see (JS content needs Selenium).
- x Selectors are stable (sites redesign - expect breakage).
- x 200 OK means right content (login walls serve 200 with wrong page).

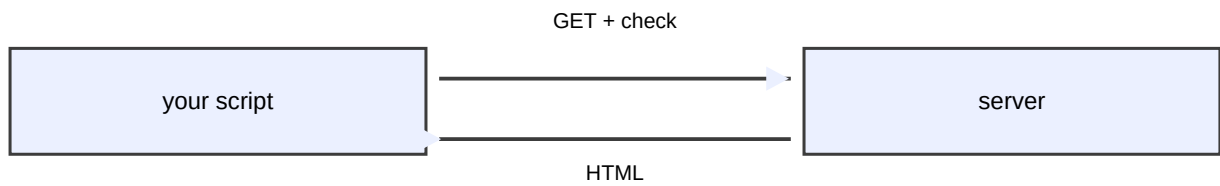
5-minute whiteboard plan

1. Download one page and print its title live.
2. Select all links; extract hrefs.
3. Students build the mapit URL from argv.
4. Exit ticket: which selenium call types text? (send_keys).

Visualize it - draw these from memory

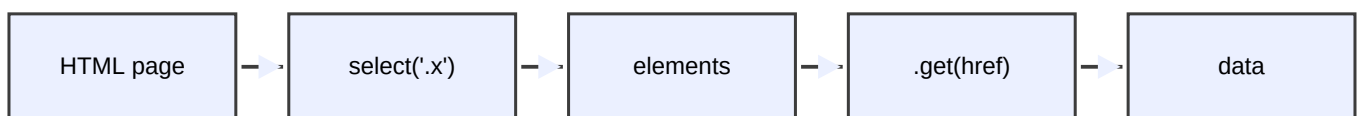
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - request goes out, page comes back



get -> raise_for_status -> parse. A 200 can still be a login wall - verify content.

Visual 2 - soup funnel: page to elements to data



CSS selectors are the funnel. When sites redesign, the funnel mouth moves - expect breakage.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Page title grabber

```
import requests, bs4
res = requests.get('https://example.com', timeout=10)
res.raise_for_status()
soup = bs4.BeautifulSoup(res.text, 'html.parser')
print(soup.title.getText())
# EXPECTED: Example Domain
```

X2. Link harvester

```
import requests, bs4
res = requests.get('https://example.com', timeout=10)
soup = bs4.BeautifulSoup(res.text, 'html.parser')
print([a.get('href') for a in soup.select('a')])
# EXPECTED: List of hrefs
```

X3. Status checker

```
import requests
for url in ['https://example.com', 'https://example.com/nope']:
    try:
        r = requests.get(url, timeout=10)
        r.raise_for_status()
        print(url, 'OK')
    except Exception as e:
        print(url, 'FAIL')
# EXPECTED: OK / FAIL lines
```

Program map - all 34 at a glance

```
01 01_project_mapit.py.py
02 02_step_1_figure.py
03 03_step_2_handle.py
04 04_step_2_handle_mapit.py
05 05_step_2_handle_mapit_2.py
06 06_step_3_handle.py
07 07_downloading_files_from.py
08 08_downloading_web_page.py
09 09_checking_errors.py
10 10_checking_errors_requests.py
11 11_checking_errors_there.py
12 12_saving_downloaded_files.py
13 13_quick_refresher.py
14 14_quick_refresher_al.py
15 15_parsing_html_beautifulsoup.py
16 16_creating_beautifulsoup_object.py
17 17_creating_beautifulsoup_object_examplefil.py
18 18_finding_element_select.py
19 19_finding_element_select_pelems.py
20 20_getting_data_from.py
21 21_step_1_get.py
22 22_step_2_find.py
23 23_step_3_open.py
24 24_step_1_design.py
25 25_step_2_download.py
26 26_step_3_find.py
27 27_step_4_save.py
28 28_step_4_save_downloading.py
29 29_starting_selenium_controlled.py
30 30_finding_elements_page.py
```

31 31_finding_elements_page_found.py
32 32_clicking_page.py
33 33_filling_out_submitting.py
34 34_sending_special_keys.py

Topic: Project: mapit.py with the webbrowser Module

01/34 01_project_mapit_py.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import webbrowser
webbrowser.open('http://inventwithpython.com/')
```

Topic: Step 1: Figure Out the URL

02/34 02_step_1_figure.py [script]

Script: save and run; predict the output first, then compare.

```
C:\> mapit 870 Valencia St, San Francisco, CA 94110
```

Topic: Step 2: Handle the Command Line Arguments

03/34 03_step_2_handle.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mapIt.py - Launches a map in the browser using an address from the
# command line or clipboard.

import webbrowser, sys
if len(sys.argv) > 1:
    # Get address from command line.
    address = ' '.join(sys.argv[1:])

# TODO: Get address from clipboard.
```

04/34 04_step_2_handle_mapit.py [script]

Script: save and run; predict the output first, then compare.

```
mapit 870 Valencia St, San Francisco, CA 94110
```

05/34 05_step_2_handle_mapit_2.py [script]

Script: save and run; predict the output first, then compare.

```
['mapIt.py', '870', 'Valencia', 'St, ', 'San', 'Francisco, ', 'CA', '94110']
```

Topic: Step 3: Handle the Clipboard Content and Launch the Browser

06/34 06_step_3_handle.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# mapIt.py - Launches a map in the browser using an address from the
# command line or clipboard.

import webbrowser, sys, pyperclip
if len(sys.argv) > 1:
    # Get address from command line.
    address = ' '.join(sys.argv[1:])
else:
    # Get address from clipboard.
    address = pyperclip.paste()

webbrowser.open('https://www.google.com/maps/place/' + address)
```

Topic: Downloading Files from the Web with the requests Module

07/34 07_downloading_files_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import requests
```

Topic: Downloading a Web Page with the requests.get() Function

08/34 08_downloading_web_page.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import requests
res = requests.get('https://automatetheboringstuff.com/files/rj.txt')
type(res)
# OUT:    <class 'requests.models.Response'>
res.status_code == requests.codes.ok
# OUT:    True
len(res.text)
# OUT:    178981
print(res.text[:250])
# OUT:    The Project Gutenberg EBook of Romeo and Juliet, by William Shakespeare

# OUT:    This eBook is for the use of anyone anywhere at no cost and with
# OUT:    almost no restrictions whatsoever. You may copy it, give it away or
# OUT:    re-use it under the terms of the Proje
```

Topic: Checking for Errors

09/34 09_checking_errors.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

res = requests.get('http://inventwithpython.com/page_that_does_not_exist')
res.raise_for_status()
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#138>", line 1, in <module>
# OUT:     res.raise_for_status()
# OUT:   File "C:\Python34\lib\site-packages\requests\models.py", line 773, in raise_for_status
# OUT:     raise HTTPError(http_error_msg, response=self)
# OUT: requests.exceptions.HTTPError: 404 Client Error: Not Found
```

10/34 10_checking_errors_requests.py [script]

Script: save and run; predict the output first, then compare.

```
import requests
res = requests.get('http://inventwithpython.com/page_that_does_not_exist')
try:
    res.raise_for_status()
except Exception as exc:
    print('There was a problem: %s' % (exc))
```

11/34 11_checking_errors_there.py [script]

Script: save and run; predict the output first, then compare.

There was a problem: 404 Client Error: Not Found

Topic: Saving Downloaded Files to the Hard Drive

12/34 12_saving_downloaded_files.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
import requests
res = requests.get('https://automatetheboringstuff.com/files/rj.txt')
res.raise_for_status()
playFile = open('RomeoAndJuliet.txt', 'wb')
for chunk in res.iter_content(100000):
# OUT:         playFile.write(chunk)

# OUT: 100000
# OUT: 78981
playFile.close()
```

Topic: A Quick Refresher

13/34 13_quick_refresher.py [script]

Script: save and run; predict the output first, then compare.

```
<strong>Hello</strong> world!
```

14/34 14_quick_refresher_al.py [script]

Script: save and run; predict the output first, then compare.

```
Al's free <a href="http://inventwithpython.com">Python books</a>.
```

Topic: Parsing HTML with the BeautifulSoup Module

15/34 15_parsing_html_beautifulsoup.py [script]

Script: save and run; predict the output first, then compare.

```
<!-- This is the example.html example file. -->

<html><head><title>The Website Title</title></head>
<body>
<p>Download my <strong>Python</strong> book from <a href="http://
inventwithpython.com">my website</a>.</p>
<p class="slogan">Learn Python the easy way!</p>
<p>By <span id="author">Al Sweigart</span></p>
</body></html>
```

Topic: Creating a BeautifulSoup Object from HTML

16/34 16_creating_beautifulsoup_object.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import requests, bs4
res = requests.get('http://nostarch.com')
res.raise_for_status()
noStarchSoup = bs4.BeautifulSoup(res.text)
type(noStarchSoup)
# OUT: <class 'bs4.BeautifulSoup'>
```

17/34 17_creating_beautifulsoup_object_examplefil.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
exampleFile = open('example.html')  
exampleSoup = bs4.BeautifulSoup(exampleFile)  
type(exampleSoup)  
# OUT: <class 'bs4.BeautifulSoup'>
```

Topic: Finding an Element with the select() Method

18/34 18_finding_element_select.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
import bs4
exampleFile = open('example.html')
exampleSoup = bs4.BeautifulSoup(exampleFile.read())
elems = exampleSoup.select('#author')
type(elems)
# OUT: <class 'list'>
len(elems)
# OUT: 1
type(elems[0])
# OUT: <class 'bs4.element.Tag'>
elems[0].getText()
# OUT: 'Al Sweigart'
str(elems[0])
# OUT: '<span id="author">Al Sweigart</span>'
elems[0].attrs
# OUT: {'id': 'author'}
```

19/34 19_finding_element_select_pelems.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

pElems = exampleSoup.select('p')
str(pElems[0])
# OUT: '<p>Download my <strong>Python</strong> book from <a href="http://
# OUT: inventwithpython.com">my website</a>.</p>'
pElems[0].getText()
# OUT: 'Download my Python book from my website.'
str(pElems[1])
# OUT: '<p class="slogan">Learn Python the easy way!</p>'
pElems[1].getText()
# OUT: 'Learn Python the easy way!'
str(pElems[2])
# OUT: '<p>By <span id="author">Al Sweigart</span></p>'
pElems[2].getText()
# OUT: 'By Al Sweigart'
```

Topic: Getting Data from an Element's Attributes

20/34 20_getting_data_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
import bs4
soup = bs4.BeautifulSoup(open('example.html'))
spanElem = soup.select('span')[0]
str(spanElem)
# OUT: '<span id="author">Al Sweigart</span>'
spanElem.get('id')
# OUT: 'author'
spanElem.get('some_nonexistent_addr') == None
# OUT: True
spanElem.attrs
# OUT: {'id': 'author'}
```

Topic: Step 1: Get the Command Line Arguments and Request the Search Page

21/34 21_step_1_get.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# lucky.py - Opens several Google search results.

import requests, sys, webbrowser, bs4

print('Googling...') # display text while downloading the Google page
res = requests.get('http://google.com/search?q=' + ' '.join(sys.argv[1:]))
res.raise_for_status()

# TODO: Retrieve top search result links.

# TODO: Open a browser tab for each result.
```

Topic: Step 2: Find All the Results

22/34 22_step_2_find.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# lucky.py - Opens several google search results.

import requests, sys, webbrowser, bs4

--snip--

# Retrieve top search result links.
soup = bs4.BeautifulSoup(res.text)

# Open a browser tab for each result.
linkElems = soup.select('.r a')
```

Topic: Step 3: Open Web Browsers for Each Result

23/34 23_step_3_open.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# lucky.py - Opens several google search results.

import requests, sys, webbrowser, bs4

--snip--

# Open a browser tab for each result.
linkElems = soup.select('.r a')
numOpen = min(5, len(linkElems))
for i in range(numOpen):
    webbrowser.open('http://google.com' + linkElems[i].get('href'))
```

Topic: Step 1: Design the Program

24/34 24_step_1_design.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# downloadXkcd.py - Downloads every single XKCD comic.

import requests, os, bs4

url = 'http://xkcd.com'          # starting url
os.makedirs('xkcd', exist_ok=True) # store comics in ./xkcd
while not url.endswith('#'):
    # TODO: Download the page.

    # TODO: Find the URL of the comic image.

    # TODO: Download the image.

    # TODO: Save the image to ./xkcd.

    # TODO: Get the Prev button's url.

print('Done.')
```

Topic: Step 2: Download the Web Page

25/34 25_step_2_download.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# downloadXkcd.py - Downloads every single XKCD comic.

import requests, os, bs4

url = 'http://xkcd.com'          # starting url
os.makedirs('xkcd', exist_ok=True) # store comics in ./xkcd
while not url.endswith('#'):
    # Download the page.
    print('Downloading page %s...' % url)
    res = requests.get(url)
    res.raise_for_status()

    soup = bs4.BeautifulSoup(res.text)

    # TODO: Find the URL of the comic image.

    # TODO: Download the image.

    # TODO: Save the image to ./xkcd.

    # TODO: Get the Prev button's url.

print('Done.')
```

Topic: Step 3: Find and Download the Comic Image

26/34 26_step_3_find.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# downloadXkcd.py - Downloads every single XKCD comic.

import requests, os, bs4

--snip--

# Find the URL of the comic image.
comicElem = soup.select('#comic img')
if comicElem == []:
    print('Could not find comic image.')
else:
    try:
        comicUrl = 'http:' + comicElem[0].get('src')
        # Download the image.
        print('Downloading image %s...' % (comicUrl))
        res = requests.get(comicUrl)
        res.raise_for_status()
    except requests.exceptions.MissingSchema:
        # skip this comic
        prevLink = soup.select('a[rel="prev"]')[0]
        url = 'http://xkcd.com' + prevLink.get('href')
        continue

# TODO: Save the image to ./xkcd.

# TODO: Get the Prev button's url.

print('Done.')
```

Topic: Step 4: Save the Image and Find the Previous Comic

27/34 27_step_4_save.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# downloadXkcd.py - Downloads every single XKCD comic.

import requests, os, bs4

--snip--

    # Save the image to ./xkcd.
    imageFile = open(os.path.join('xkcd', os.path.basename(comicUrl)), 'wb')
    for chunk in res.iter_content(100000):
        imageFile.write(chunk)
    imageFile.close()

# Get the Prev button's url.
prevLink = soup.select('a[rel="prev"]')[0]
url = 'http://xkcd.com' + prevLink.get('href')

print('Done.')
```

28/34 28_step_4_save_downloading.py [script]

Script: save and run; predict the output first, then compare.

```
Downloading page http://xkcd.com...
Downloading image http://imgs.xkcd.com/comics/phone_alarm.png...
Downloading page http://xkcd.com/1358/...
Downloading image http://imgs.xkcd.com/comics/nro.png...
Downloading page http://xkcd.com/1357/...
Downloading image http://imgs.xkcd.com/comics/free_speech.png...
Downloading page http://xkcd.com/1356/...
Downloading image http://imgs.xkcd.com/comics/orbital_mechanics.png...
Downloading page http://xkcd.com/1355/...
Downloading image http://imgs.xkcd.com/comics/airplane_message.png...
Downloading page http://xkcd.com/1354/...
Downloading image http://imgs.xkcd.com/comics/heartbleed_explanation.png...
--snip--
```

Topic: Starting a Selenium-Controlled Browser

29/34 29_starting_selenium_controlled.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from selenium import webdriver
browser = webdriver.Firefox()
type(browser)
# OUT: <class 'selenium.webdriver.firefox.webdriver.WebDriver'>
browser.get('http://inventwithpython.com')
```

Topic: Finding Elements on the Page

30/34 30_finding_elements_page.py [script]

Script: save and run; predict the output first, then compare.

```
from selenium import webdriver
browser = webdriver.Firefox()
browser.get('http://inventwithpython.com')
try:
    elem = browser.find_element_by_class_name('bookcover')
    print('Found <%s> element with that class name!' % (elem.tag_name))
except:
    print('Was not able to find an element with that name.')
```

31/34 31_finding_elements_page_found.py [script]

Script: save and run; predict the output first, then compare.

Found element with that class name!

Topic: Clicking the Page

32/34 32_clicking_page.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
from selenium import webdriver  
browser = webdriver.Firefox()  
browser.get('http://inventwithpython.com')  
linkElem = browser.find_element_by_link_text('Read It Online')  
type(linkElem)  
# OUT: <class 'selenium.webdriver.remote.webelement.WebElement'>  
linkElem.click() # follows the "Read It Online" link
```

Topic: Filling Out and Submitting Forms

33/34 33_filling_out_submitting.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from selenium import webdriver
browser = webdriver.Firefox()
browser.get('https://mail.yahoo.com')
emailElem = browser.find_element_by_id('login-username')
emailElem.send_keys('not_my_real_email')
passwordElem = browser.find_element_by_id('login-passwd')
passwordElem.send_keys('12345')
passwordElem.submit()
```

Topic: Sending Special Keys

34/34 34_sending_special_keys.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
browser = webdriver.Firefox()
browser.get('http://nostarch.com')
htmlElem = browser.find_element_by_tag_name('html')
htmlElem.send_keys(Keys.END)      # scrolls to bottom
htmlElem.send_keys(Keys.HOME)    # scrolls to top
```